

The Hut

Language, Repair, and the Unfinished Program

Watson Hartsoe

Working Paper — 17 August 2026

If language is a house, it is not yet Heidegger’s house of Being, and it is not yet Turing’s mansion for a soul. It is a hut with one wall leaning out, a roof patched twice, a door that no longer meets the floor, and a bucket under the place where rain discovers what the drawing forgot. Someone is already inside. Someone else has already repaired it. The poles have histories. The ground has owners, ghosts, roots, runoff, and previous holes. The first sentence arrives late.

Natural-language programming is usually staged somewhere much cleaner. A blank box waits. A user says what should exist. A model answers with code, geometry, text, behavior, or a world. The result is inspected, corrected, and generated again. It is tempting to say that the prompt begins the program and iteration completes its specification. But a blank box is not empty ground. Models arrive full of inherited regularities. Software arrives with libraries, defaults, interfaces, operating systems, APIs, permissions, habits, and technical debt. Users arrive with examples they have seen, metaphors they trust, distinctions they have not yet learned to make, and workarounds they no longer notice as workarounds. Before a single instruction is typed, much of the house is already standing.

The hut is useful precisely because it is too poor to conceal this fact. A mansion can hide its maintenance in servants’ corridors, service contracts, foundations, ducts, warranties, and walls thick enough to make the labor of keeping it alive disappear. A hut leaves the patch in view. It is less a primitive architecture than an architecture whose unfinishedness has nowhere to hide.

Ground

Marc-Antoine Laugier’s primitive hut is one of architecture’s most elegant acts of cleaning. In *An Essay on Architecture*, upright trunks become columns, horizontal members become entablatures, sloping branches become the roof. The rustic cabin is made to stand at the beginning so that architecture can recover its essential elements from an imagined scene before ornament, corruption, and historical complication [6]. The hut serves as an origin because it removes almost everything that makes an actual beginning impossible.

Tim Ingold attacks precisely this search for a first hut. In his account of building, dwelling, and living, the desire to locate a point at which architecture, history, or humanity properly begins is not an innocent archaeological question. It is produced by a conceptual picture in which design comes first, building follows, and dwelling starts only after the building is finished [4]. Once that sequence is accepted, the mind naturally goes looking for the first design, the first construction, the first completed shelter.

But there is no first post in the sense Laugier requires. A post is cut with an inherited tool, according to a learned technique, from a species of wood whose growth already records soil and

weather. It is set into ground whose hardness, slope, moisture, ownership, and previous use immediately constrain what can happen next. Even the most improvised shelter begins by entering arrangements already underway.

Programming has its own primitive hut fantasy: the first prompt. A clean input appears to precede the system that follows. Yet the first prompt is already an intervention in a dense technical settlement. The model has been trained. The libraries have been written. The platform has already decided what counts as a file, an image, an error, a safe request, a network call, a user, a session, a memory, a tool. The prompt does not enter emptiness. It enters a city pretending to be a clearing.

This matters because specifications often inherit the same fiction. Requirements are treated as if they precede implementation and merely await faithful realization. Sometimes they do. A cryptographic primitive, a flight-control law, or a dosage limit may need to be fixed before execution. But much ordinary making begins inside a condition where what counts as relevant cannot yet be fully separated from what has already been built. The ground itself teaches distinctions. A wet patch says something the site plan did not. A dependency conflict reveals a boundary the user did not know existed. A generated room exposes a sightline that the word “private” had not articulated. The supposed beginning is already a reply.

Keep the hut. Remove its firstness.

Posts

A hut stands because its parts hold one another, but rules do not hold themselves up.

Ingold’s medieval masons worked with constructive geometry, templates, rules of thumb, and carefully taught procedures. None of this makes their practice ruleless. What it does is deny the stronger fantasy that rules contain the work in advance. The rules were, in his formulation, “resources for action” rather than determinants of action [5]. Medieval stereotomy combined prescribed steps with judgement, experience, and “creative extemporisation.” The mason still had to look, fit, cut, file, reject, and try again.

The difference is easy to blur because formal systems can genuinely generate architecture. Stiny and Mitchell’s Palladian grammar shows that architectural canons and exemplar plans can be recast as a parametric shape grammar capable of generating Palladian ground plans [9]. That achievement matters. It proves that some architectural knowledge can be made operational as explicit transformations over representations. But it also clarifies what has been achieved: a formal interpreter applies rules to shapes and produces geometry. The grammar does not quarry stone, notice a crack, compensate for a warped timber, negotiate with a patron, smell damp, or decide that the rule should be bent because the wall already standing is six centimeters out.

There are therefore at least two ways a rule can live in a house. It can determine a transformation in a formal space. Or it can orient skilled action in a material situation that outruns the rule. Both are real. Confusing them creates the familiar error in which a compact specification is credited with powers exercised by interpreters, tools, workers, institutions, materials, and environments.

A natural-language instruction lives awkwardly between these two forms. “Make the bedroom private” is not a formal transformation rule. It does not say where the wall goes, how privacy is measured, whether acoustic separation matters, whether a sightline counts through reflected glass, whether a door may align with an entrance, or whether privacy is individual, familial, visual, legal, or cultural. Yet it is not therefore empty. It can orient action. A model interprets it. A designer recognizes a violation. A resident closes a curtain. A later correction states “no direct sightline from entrance to bed.” What begins as guidance may harden into a constraint after contact with a

particular arrangement.

The dangerous move is to describe that hardening as if the constraint had simply been hiding inside the original word all along. “Private” did not contain one secret floor plan waiting to be decoded. The generated arrangement, the observer, and the situation helped decide which meaning became consequential. A post is not merely located by a drawing; once planted, it changes where the next post can go.

Walls

Turing’s theological objection contains one of computing’s strangest architectural metaphors. If machine-makers cannot create souls, he suggests, perhaps they can still be instruments of divine will by “providing mansions for the souls that He creates” [11]. The argument is deliberately speculative. Its usefulness lies less in the theology than in the clean distinction the image produces: a maker constructs the mansion; something else may inhabit it.

That distinction is extraordinarily attractive in computing. Hardware is the mansion, software the inhabitant. The operating system is the mansion, the application its tenant. A virtual world is the mansion, an agent its inhabitant. The program constructs a possibility-space, and behavior takes place inside it. The architecture can be attributed to one causal layer while agency is attributed to another.

But not every house permits this separation.

Suzanne Preston Blier’s account of Batammariba architecture records a very different relation among house, body, kinship, ritual, and personhood [1]. House parts correspond to bodily parts; earthen fabric and plaster are described through bodily analogies; houses participate in ritual events and life histories; spiritual components may be attributed to houses as well as persons. Most disruptive to the simple container model is the reported term *takyeta*, which can denote both “house” and “family.” The house is not merely an empty shell occupied by a social unit that was already fully itself elsewhere.

No software ontology follows automatically from this ethnography. The point is not to romanticize another architecture into a universal theory of computation. The point is to lose confidence in the obviousness of the mansion. Substrate and inhabitant are sometimes separable; sometimes the relation partly makes the things being separated.

Long-lived technical systems often behave more like this than the mansion metaphor admits. Users train routines around interfaces; interfaces change in response to observed use; datasets accumulate the traces of those routines; extensions and workarounds become indispensable; organizations rewrite policy around what the software makes easy or measurable; the software is then redesigned around the organization it helped produce. The system houses activity, but the activity also rebuilds the system. After enough cycles, asking which is the house and which is the inhabitant becomes less informative than asking who can change the wall.

This is where language becomes architectural in a less glamorous sense. Names assign rooms. Categories decide who belongs where. A permission becomes a door. A default becomes a corridor. A schema becomes a load-bearing wall because everything else is built against it. None of these structures need be physical to constrain motion. Nor are they merely metaphors once changing the category changes what actions a system permits.

The house of language is not made only from sentences. It is made from every distinction that must be accepted before the next action becomes possible.

Roof

The roof is where the clean drawing meets weather.

Ingold's account of architectural completion becomes most concrete when rain enters it. A finished building is supposed to stabilize the architect's design. But buildings remain in a world of growth, decay, wind, rot, insects, water, and maintenance. "Completion is, at best, a legal fiction," he writes; the serious work may begin only when occupants inherit the building and start dealing with everything the formal design kept outside [5]. Frank Lloyd Wright's Fallingwater appears in this discussion as the famous "seven-bucket building." The bucket under the drip is not part of the architectural conception. It is the resident's small prosthesis for keeping the conception habitable.

The leak changes the problem of correctness. A roof can look correct in a rendering and fail in rain. It can pass a geometric test and fail after leaves clog the gutter. It can survive one storm and rot after five winters. Failure is therefore not simply a property waiting visibly inside an artifact. Failure appears in a relation among artifact, environment, duration, observer, and use.

The same thing happens in generated software. A page can look right and be inaccessible to a screen reader. A function can pass examples and fail under concurrency. A data pipeline can produce the expected table while silently leaking private information. A generated script can work on the author's machine because the environment supplies an undeclared dependency. A chatbot can seem helpful until repeated use shifts the burden of correction onto the same people whose circumstances its categories fail to represent.

The lesson is not that inspection is useless. It is that every inspection has weather. A test environment decides which rain is allowed to fall.

The more seductive claim is that failure can simply become specification: observe the leak, add the constraint, regenerate. Counterexample-guided synthesis gives this idea a rigorous computational form. A candidate is proposed; a validator finds a counterexample; the counterexample enlarges the evidence constraining the next candidate. But the roof introduces dirt that formal verification does not remove. Who notices the failure? How long does it take to appear? Who is exposed while it appears? Can it be reproduced? Does the system have an oracle for it at all? Does the person with the bucket have the authority to change the roof?

A leak is information only for someone in a position to make it count.

The bucket

The bucket is the smallest object in the hut and the one most likely to disappear from a system diagram.

A workaround has two opposed meanings. It reveals that the structure does not fit the world, and it allows the structure to keep operating without being changed. The bucket makes the leak legible. It also lets the roof remain unfixed for another night.

Software lives on buckets. Users reformat exports by hand. Operators restart services at dawn. Analysts maintain shadow spreadsheets because the official database cannot represent the case they actually have. Moderators memorize exceptions. Disabled users invent navigation tricks around inaccessible interfaces. Programmers pin old package versions because an upgrade breaks a dependency chain no one has time to repair. Prompt writers learn incantations that compensate for a model's recurring failure and then pass those incantations around as expertise.

None of this makes the system nonfunctional. Quite the opposite. The labor is often the reason it appears functional.

This creates a hard distinction between dwelling and abandonment. Ingold's dwelling perspective rightly refuses the idea that use begins only after making ends. Residents alter, maintain, improvise, and repair. Suchman similarly showed why plans cannot exhaust situated action: actual action is organized in circumstances that plans cannot fully anticipate [10]. But the dignity of situated intelligence can be turned into an alibi for bad construction. "People will adapt" can mean that people will be forced to finish the work without authority, credit, compensation, or the power to alter the conditions that produce the problem.

The decisive question is not whether users modify a system. They always do. It is whether their modifications count as participation in making or as exported maintenance burden.

The distinction cannot be read directly from the existence of repair. A resident repainting a wall may be appropriation. A nurse rechecking every automatically generated dosage may be unpaid safety infrastructure. A programmer extending an open system may be authorship. A customer repeatedly rephrasing a request because the interface refuses their dialect may be doing compensatory labor. The same outward pattern—user changes something after handoff—can conceal very different distributions of power.

Who carries the bucket? Who owns the roof? Who can authorize the repair? Who pays for the damage while the repair is pending? Who gets to call the building complete?

These are programming questions once computation becomes something people must inhabit rather than merely operate.

Holes

Formal synthesis offers a cleaner kind of incompleteness. In program sketching, a partial program can contain explicit holes whose values or fragments are synthesized subject to tests or a reference implementation [8]. This is one of the great virtues of formal representation: uncertainty becomes local. Here is what is fixed. Here is what remains open. Search here.

A hut rarely grants this luxury.

The problem is not that there are no gaps. It is that the gaps do not arrive already outlined. An inherited wall may occupy the place where the real uncertainty is. A default may be mistaken for a requirement. A database column may make a social category look natural because every downstream component now expects it. A model may produce a stereotyped house in response to "home," making some possibilities concrete and leaving others so unrepresented that the user never thinks to reject them.

The generated artifact therefore does more than fill known holes. It proposes where the holes are.

Suppose a user asks for "a private bedroom." The model places a bed, a door, windows, walls, circulation, furniture, and perhaps an entrance. The result may reveal a direct sightline from front door to bed. The user then says, "No direct sightline from entrance to bed." That correction looks like specification refinement. Yet it also records the history of one proposal. Had the first artifact instead put the bedroom upstairs, the user might have discovered an acoustic problem first. Had it used a courtyard, the user might have cared about exposure to neighboring windows. The sequence of generated artifacts changes which requirements become thinkable soon enough to be written down.

This is not mystical co-creation. It is path dependence. Every concrete proposal makes some distinctions cheap to notice and others expensive to imagine.

A good programming environment should therefore mark inherited answers as aggressively as sketching marks holes. It should be able to say: this wall came from your instruction; this one came

from a library default; this one was inferred by the model; this one survived from an earlier version; this one is required by regulation; this one exists only because five other things now depend on it. Without such provenance, the system presents a settlement as if it were a clearing.

The hardest question is not “what is missing?” It is “who got to make the missingness local?”

Patchwork

A cathedral is too large to call a hut, but Chartres keeps the hut’s secret visible if one looks closely enough.

Ingold, following David Turnbull and John James, describes the rebuilding of Chartres after the fire of 1194 not as the transparent execution of a surviving master plan but as an episodic construction across many campaigns, teams, and master masons [5]. The evidence does not prove that no prior plans existed. That uncertainty matters. The stronger point is that prior design cannot explain the work by itself. Plans do not transmute into stone. Workmanship, templates, communication, adjustment, and local judgement carry continuity across changing hands.

Turnbull’s criticism of the design argument is exact: it “explains too little and too much.” It explains too little when it skips the skilled activity required to realize any design. It explains too much when it compensates by attributing to codified rules powers the rules cannot exercise.

The visible harmony of Chartres therefore cannot be taken as proof that a complete prior specification caused it. Closer inspection reveals irregularly disposed and imperfectly matched elements, an ad hoc accumulation whose continuity was carried through practice and communication. The magnificent whole did not erase the patchwork. It made the patchwork easier to forget.

Software acquires the same false retrospect. Once a system works, its history is rewritten as architecture. The surviving modules appear necessary. The interfaces look intentional. The constraints look as though they were always requirements. Failed branches vanish. Workarounds become “process.” A sequence of local repairs hardens into something future maintainers call the design.

Peter Naur’s account of programming as theory building blocks one easy solution to this problem [7]. The program text and its documentation can survive while the theory possessed by the programmers who built and modified the system disappears. Keeping more text is therefore not equivalent to keeping understanding. A complete prompt transcript can preserve every utterance and still fail to preserve why a distinction mattered, what analogy justified a choice, which exception would have caused a different decision, or what the builders would do when the circumstances changed.

A patchwork needs memory, but memory alone does not explain the stitching.

Dependency systems such as de Kleer’s ATMS show one way to preserve more than chronology by recording assumptions, justifications, environments, and contradictions [3]. Yet even that machinery depends on the relevant claims becoming representable. “This room feels exposed,” “the route is too ceremonial,” “the system has stopped feeling like ours,” or “I can make it work but only by checking everything twice” may matter long before they can be expressed as propositions precise enough for a truth-maintenance system.

The hut survives partly because someone remembers how to notice what has not yet become a rule.

Threshold

A door is a closure that must sometimes close.

There is a danger in praising unfinishedness. A roof that is forever “becoming” can still crush someone. A medical device cannot answer every defect by appealing to emergence. A cryptographic protocol cannot outsource its proof obligations to future inhabitants. A bridge must eventually be certified for loads that were specified before anyone is allowed to test them with a crowd.

The fact that completion can be a legal fiction does not make law fictional in the trivial sense. Handover, warranty, substantial completion, safety certification, versioning, and release boundaries create real distributions of authority and liability. They are cuts through continuing activity, but cuts can be necessary. The question is not whether closure should disappear. It is what kind of closure has been achieved and what remains alive on the other side of it.

A useful closure states its weather. It says what has been tested, under which conditions, for how long, against which adversaries, with which assumptions, and with whose labor excluded from the account. It distinguishes a verified property from a tolerated defect, a known limitation from an invisible one, a user extension from a maintenance burden. It does not call the bucket part of the roof simply because the floor is dry.

This matters especially for natural-language programming because its interfaces make early closure feel easy. A user can describe a desired thing and receive something runnable in seconds. The speed of manifestation can make the distance between appearance and adequacy harder to feel. A generated program looks more finished than the understanding around it. Its syntax may be complete while its conditions of use remain almost entirely unsettled.

The threshold should therefore be marked, not worshipped. Version 1.0 is a door, not the end of the house.

Smoke

A hut is known from its smoke before its architecture is understood. Something is happening inside.

Richard Brautigan’s “All Watched Over by Machines of Loving Grace” imagines computers no longer as objects sitting apart from nature but distributed through meadow, forest, and ecology, mammals and computers living in “mutually programming harmony” [2]. The image is seductive because the machine disappears into habitat. But once computation becomes habitat, the politics of programming change. It is one thing to put down a tool. It is another to leave the weather.

Feedback is not mutuality. A system can adapt continuously to inhabitants while the inhabitants have no power to inspect what is inferred about them, alter the rules, refuse classification, remove their data, or exit without losing access to ordinary life. The more infrastructural computation becomes, the less adequate the programmer/program distinction becomes as a description of who acts upon whom.

Natural language intensifies this ambiguity. Words feel available to everyone, but the sentence is only the visible end of a long interpreter stack. A request may pass through a language model, generated code, a framework, an API, a database schema, a policy engine, an operating system, a network service, an institutional rule, and a human operator before anything changes in the world. At every layer the sentence can be transformed, refused, narrowed, expanded, logged, priced, or made irreversible.

How many interpreters away from the wall is a word?

Counting them is not enough. The relevant question is what each interpreter can do. Which

can change meaning? Which can stop execution? Which remembers? Which can be appealed to? Which is legally accountable? Which is invisible to the person who spoke?

The hut is made of these mediations. Language does not leap into matter. It travels through hands.

The Hut

There is no need to choose between code and language as though one were real programming and the other a vague imitation. The harder division is between structures whose conditions of action have been made explicit and those whose continuing dependence on interpretation, labor, history, and environment has been hidden.

A prompt can be precise and still inherit a bad wall. Code can be formal and still depend on a bucket. A rule can be explicit and still require skill. A house can stand and still be unfinished. A user can adapt and still be abandoned. A system can remember every correction and still forget the theory that made those corrections intelligible.

Programming begins before the prompt because the ground is already occupied. It continues after the program because inhabitants keep discovering what the program could not contain. Between those two facts lies the hut: not an origin, not a final architecture, but a provisional shelter made durable enough for the next action.

The hut has no clean boundary between specification and repair. One stone prepares the place for another. One failure makes a distinction visible. One workaround conceals the urgency of a defect. One inherited choice hardens into a wall. One inhabitant finds a door where the builder saw only structure. The work is neither free improvisation nor execution of a complete plan. It is constrained correspondence with what is already there.

That is why the house of language should be allowed to remain poor.

Before language becomes a mansion for souls, it is a roof held up by maxims that do not determine the hand, categories that do not exhaust the people living under them, and patches whose ugliness records where reality refused the plan. Its intelligence is not that it contains the world. It is that someone can still alter it when the rain comes in.

References

- [1] Blier, Suzanne Preston. 1983. "Houses Are Human: Architectural Self-Images of Africa's Tamberma." *Journal of the Society of Architectural Historians* 42(4): 371–382.
- [2] Brautigan, Richard. 1967. "All Watched Over by Machines of Loving Grace." San Francisco: The Communication Company.
- [3] de Kleer, Johan. 1986. "Problem Solving with the ATMS." *Artificial Intelligence* 28(2): 197–224.
- [4] Ingold, Tim. 2000. *The Perception of the Environment: Essays on Livelihood, Dwelling and Skill*. London and New York: Routledge. Especially Chapter 10, "Building, Dwelling, Living."
- [5] Ingold, Tim. 2013. *Making: Anthropology, Archaeology, Art and Architecture*. London and New York: Routledge. Especially Chapter 4, "On Building a House."

- [6] Laugier, Marc-Antoine. 1755. *An Essay on Architecture; in Which Its True Principles Are Explained, and Invariable Rules Proposed*. London: T. Osborne and Shipton. English translation of *Essai sur l'architecture*.
- [7] Naur, Peter. 1985. "Programming as Theory Building." *Microprocessing and Microprogramming* 15(5): 253–261.
- [8] Solar-Lezama, Armando. 2009. "The Sketching Approach to Program Synthesis." In *Programming Languages and Systems: APLAS 2009*, LNCS 5904, 4–13. Springer.
- [9] Stiny, George, and William J. Mitchell. 1978. "The Palladian Grammar." *Environment and Planning B: Planning and Design* 5(1): 5–18.
- [10] Suchman, Lucy A. 1987. *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge: Cambridge University Press.
- [11] Turing, A. M. 1950. "Computing Machinery and Intelligence." *Mind* 59(236): 433–460.